Lecture 9: Synthetic Control Methods, Part 1

James Sears*
AFRE 891/991 SS 25
Michigan State University

*Parts of these slides are adapted from **"Econometrics III"** by Ed Rubin.

Table of Contents

Prologue

Prologue

This lecture is focusing on **Synthetic Control Methods**, which will let us solve several of the issues that can affect methods we discussed last lecture.

Part 1

- The Fundamental Problem of Causal Inference
- Matching
- Canonical Synthetic Control

Part 2

- Synthetic Diff-in-Diff
  - Uniform Adoption
  - Staggered Adoption
- Partially Pooled Synthetic Control

Prologue

Packages we'll use today:

```
if (!require("pacman")) install.packages("pacman")
pacman::p_load(fixest,  tidyverse)
```

As well, let's load the event study data from **Sears et al. (2023)** we finished last lecture with:

```
sah ← readRDS("data/sah_es.rds")
```

Causal Inference

Let's chat about the **fundamental problem of causal inference** for a moment.

Consider unit i's **potential outcomes:**

- $\mathbf{Y}_{1i}$: the outcome for unit i **under the treatment**
  - Treatment assignment $D_i = 1$
- $\mathbf{Y}_{0i}$: the outcome for unit i **absent the treatment**
  - Treatment assignment $D_i = 0$

Causal Inference

- $\mathbf{Y}_{1i}$: the outcome for unit i **under the treatment**
- $\mathbf{Y}_{0i}$: the outcome for unit i **absent the treatment**

We want to know

$$\tau_i = \mathbf{Y}_{1i} - \mathbf{Y}_{0i}$$

Fundamental problem: we cannot simultaneously observe *both* $\mathbf{Y}_{1i}$ *and* $\mathbf{Y}_{0i}$.

Most (all?) empirical strategies boil to **estimating** $\mathbf{Y}_{0i}$ for treated individuals — the **unobservable counterfactual** for the treatment group.

Causal Inference

Last lecture gave an overview of regression methods that make different assumptions about that **unobservable counterfactual**.

1. RCT + Random Assignment

- The average in the control group is what the average in the treatment group would have been absent the treatment
- On average, no systematic difference in (un)observables between treatment + control
- Regress outcome on treatment dummy and you're good to go
  - Maybe add some control variables to improve precision of estimator

2. Difference-in-Differences + Event Study

- The **change over time** in the control group is what the change in the treatment group would have been absent the treatment

Causal Inference

All of these estimates are identified under a variation of the **Conditional Independence Assumption (CIA)**†

$$\{\mathrm{Y}_{0i}, \mathrm{Y}_{1i}\} \perp\!\!\!\perp \mathrm{D}_i \mid \mathbf{X}_i$$

Conditional on $\mathbf{X}_i$[1], potential outcomes $(\mathrm{Y}_{0i}, \mathrm{Y}_{1i})$ are independent of treatment status (D_i).

† AKA "selection on observables".

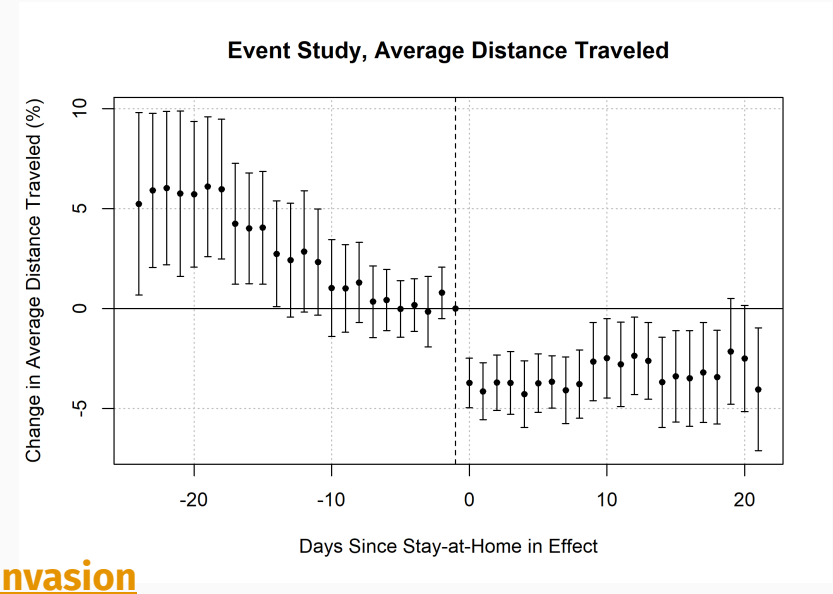
1. Or `(_{it})` if we're in a panel setting

Causal Inference

But there are times when **CIA fails**[2].

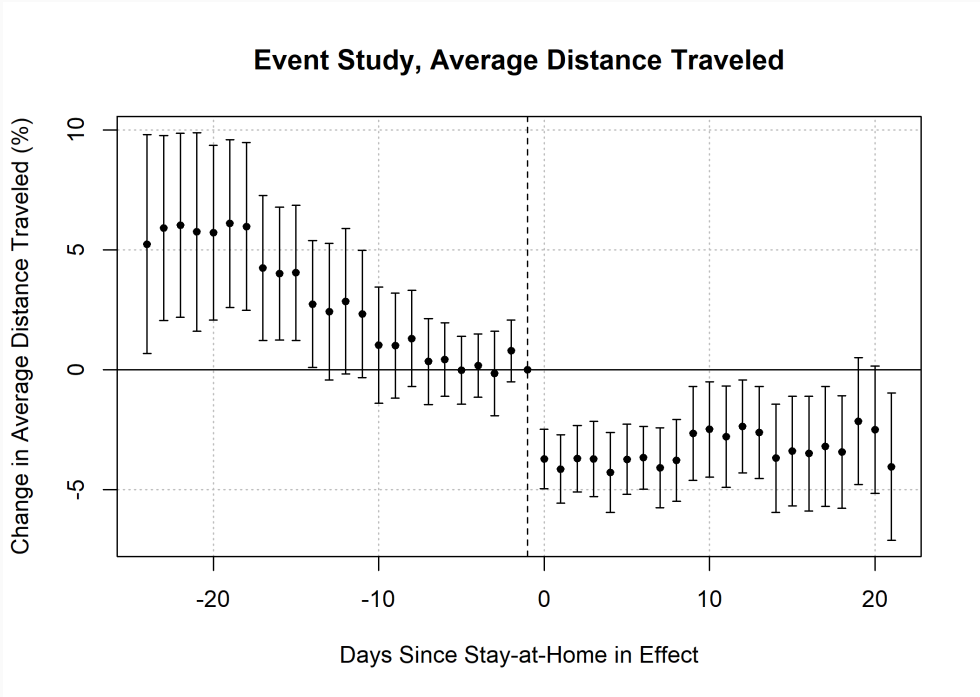
In the case of Diff-in-Diff and Event Study, this is often due to a failure of **parallel trends**.

For example, recall the event study for mobility responses to stay-at-home mandates from last lecture:



2. See the **Bay of Pigs Invasion**

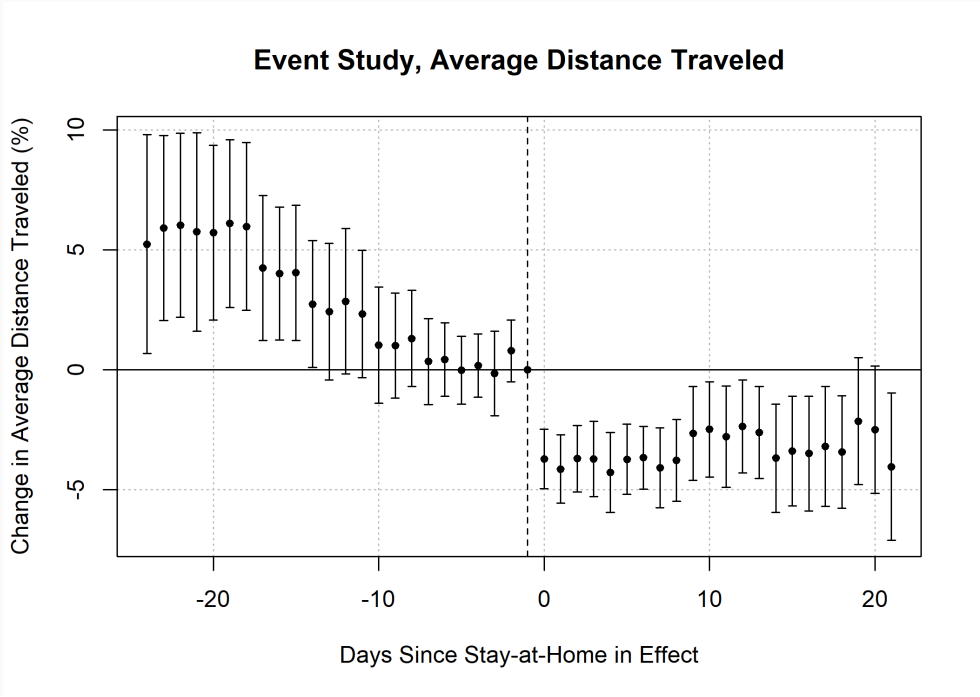
Causal Inference



In this case, the states that never adopted stay-at-home mandates might not be *valid counterfactuals* for the states that did adopt.

However, there might be a way to *construct* a valid counterfactual from the set of control units...

Causal Inference



... but before we get into that, let's chat briefly about **matching estimators**.

Matching

Matching

Matching Estimators provide an alternate way of coming up with the unobservable counterfactual for the treatment group.

The gist:

- Match untreated observations to treated observations similar on $\mathbf{X}_i$
  - *i.e.* calculate a $\widehat{\mathbf{Y}_{0i}}$ for each $\mathbf{Y}_{1i}$, based upon "matched" untreated individuals with (nearly) identical values of X_i
- If CIA holds, then we can just calculate a bunch of treatment effects conditional on $\mathbf{X}_i$
  - *i.e.*

$$\tau(x) = E[\mathbf{Y}_{1i} - \mathbf{Y}_{0i} \mid \mathbf{X}_i = x]$$

Matching (Formally)

We want to construct a counterfactual for each individual with $\mathrm{D}_i = 1$.

CIA: The counterfactual for i should **only use individuals that match** X_i.

Let there be N_T treated individuals and N_C control individuals. We want

- N_T sets of weights
- N_C weights in each set

$$w_i(j) \ (i = 1, \ldots, N_T; \ j = 1, \ldots, N_C)$$

Assume $\sum_j w_i(j) = 1$. Our estimate for the counterfactual of treated i is

$$\widehat{\mathrm{Y}_{0i}} = \sum_{j \in (D=0)} w_i(j) \mathrm{Y}_j$$

Weight for it

So all we need is those weights and we're done.

Q: Where does one find these handy weights?

A: You've got options, but you need to choose carefully/responsibly.

E.g. if $w_i(j) = \frac{1}{N_C}$ for all (i, j), then we're back to a **difference in means**.
This weighting doesn't abide by our conditional independence assumption.

The plan: choose weights $w_i(j)$ that indicate ***how close*** $\mathbf{X}_j$ is to $\mathbf{X}_i$.

Weight for it

Some common choice of weights:

- **Nearest neighbor:**

$$\mathrm{d}_{i,j} = \left(\mathrm{X}_i - \mathrm{X}_j\right)' \left(\mathrm{X}_i - \mathrm{X}_j\right)$$

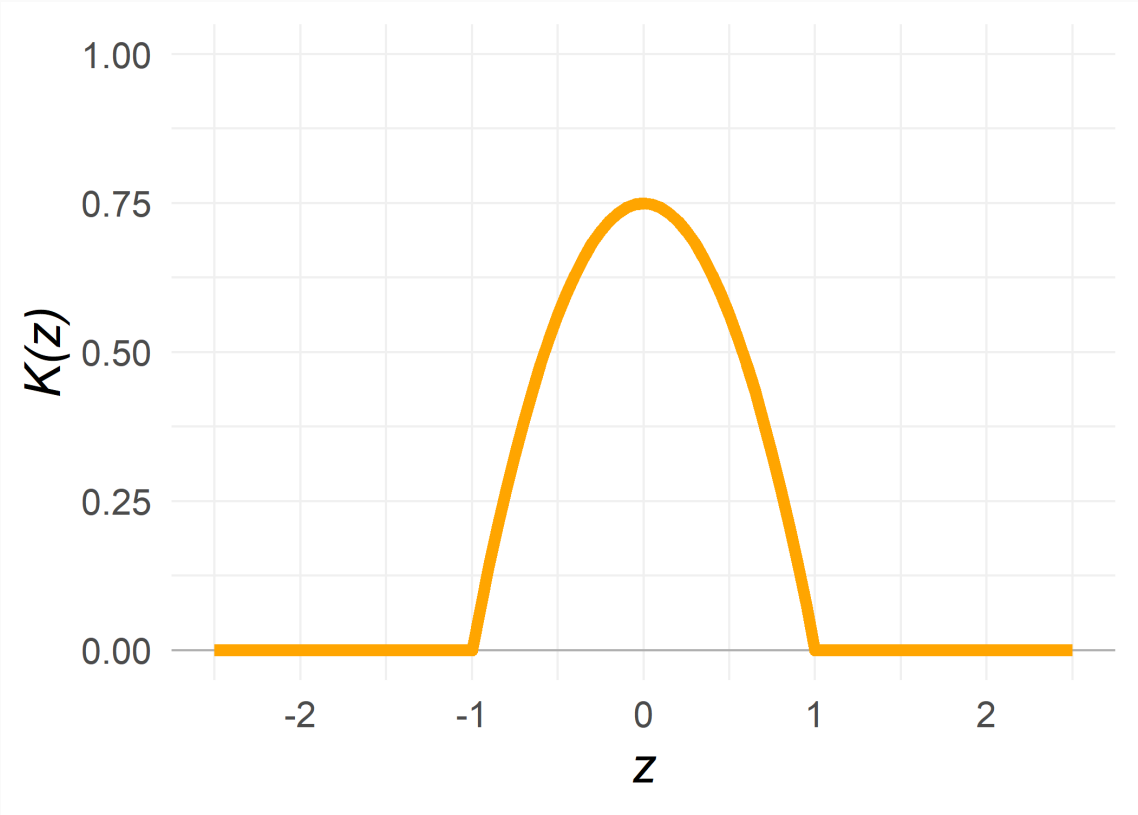
- **Kernel Matching** for **bandwidth** h and **kernel function** $K(\cdot)$:

$$w_i(j) = \frac{K\left(\frac{\mathrm{X}_j - \mathrm{X}_i}{h}\right)}{\sum_{j \in (D=0)} K\left(\frac{\mathrm{X}_j - \mathrm{X}_i}{h}\right)}$$

Kernels

For example, the **Epanechnikov kernel** is defined as

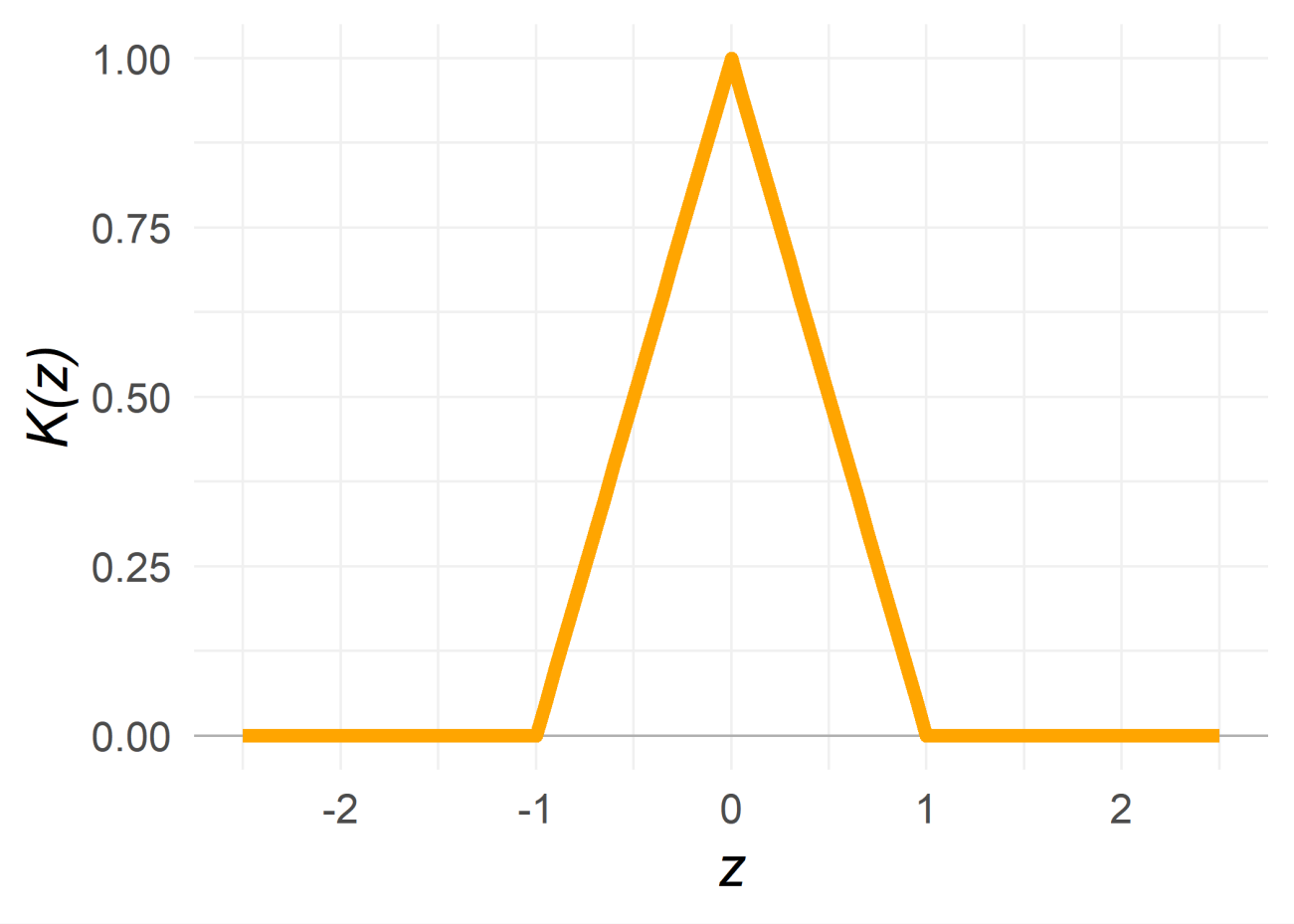
$$K(z) = \frac{3}{4}\left(1 - z^2\right) \times \mathbb{I}(|z| < 1)$$



Kernels

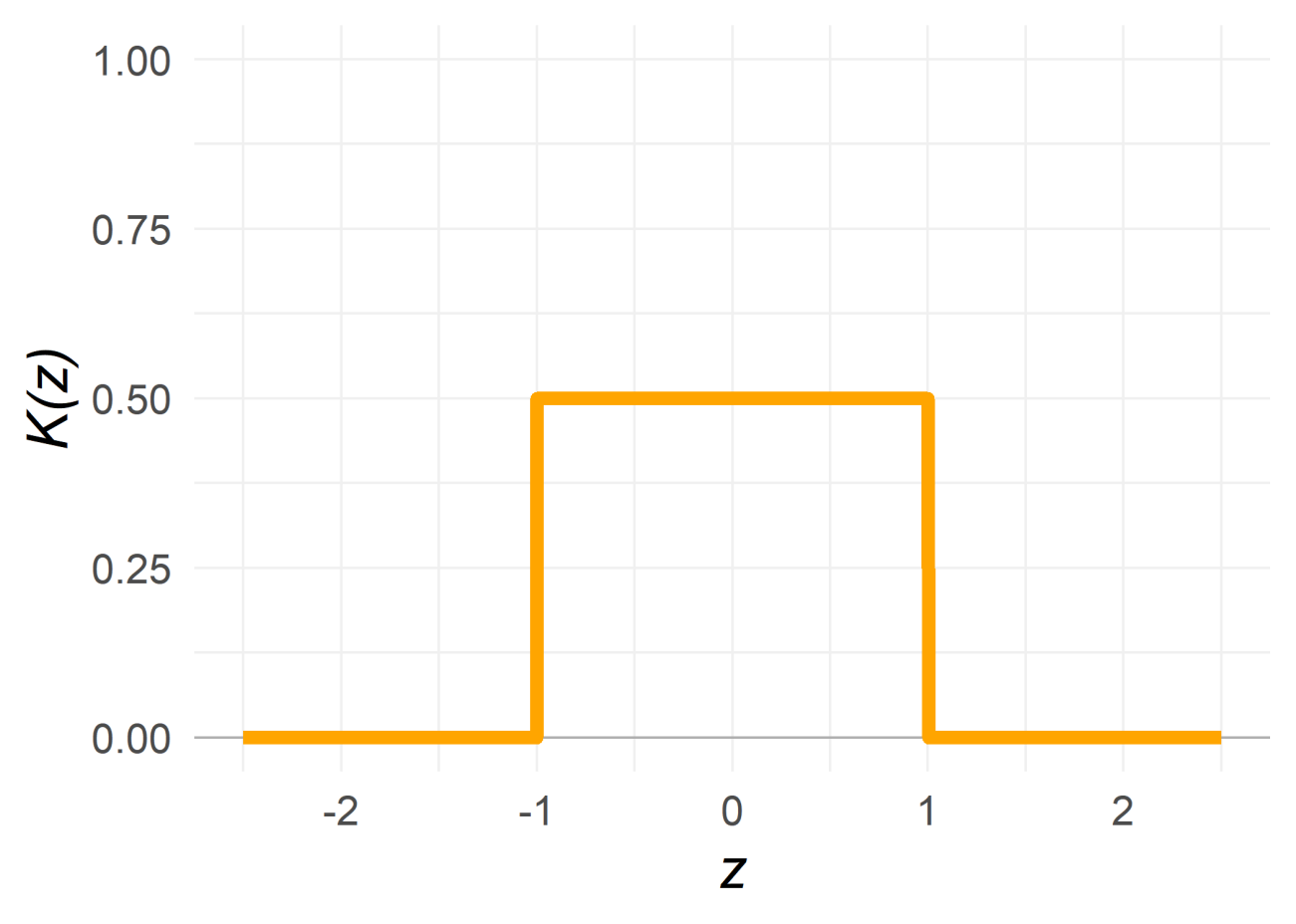
And the **triangular kernel** can be expressed as

$$K(z) = (1 - |z|) \times \mathbb{I}(|z| < 1)$$



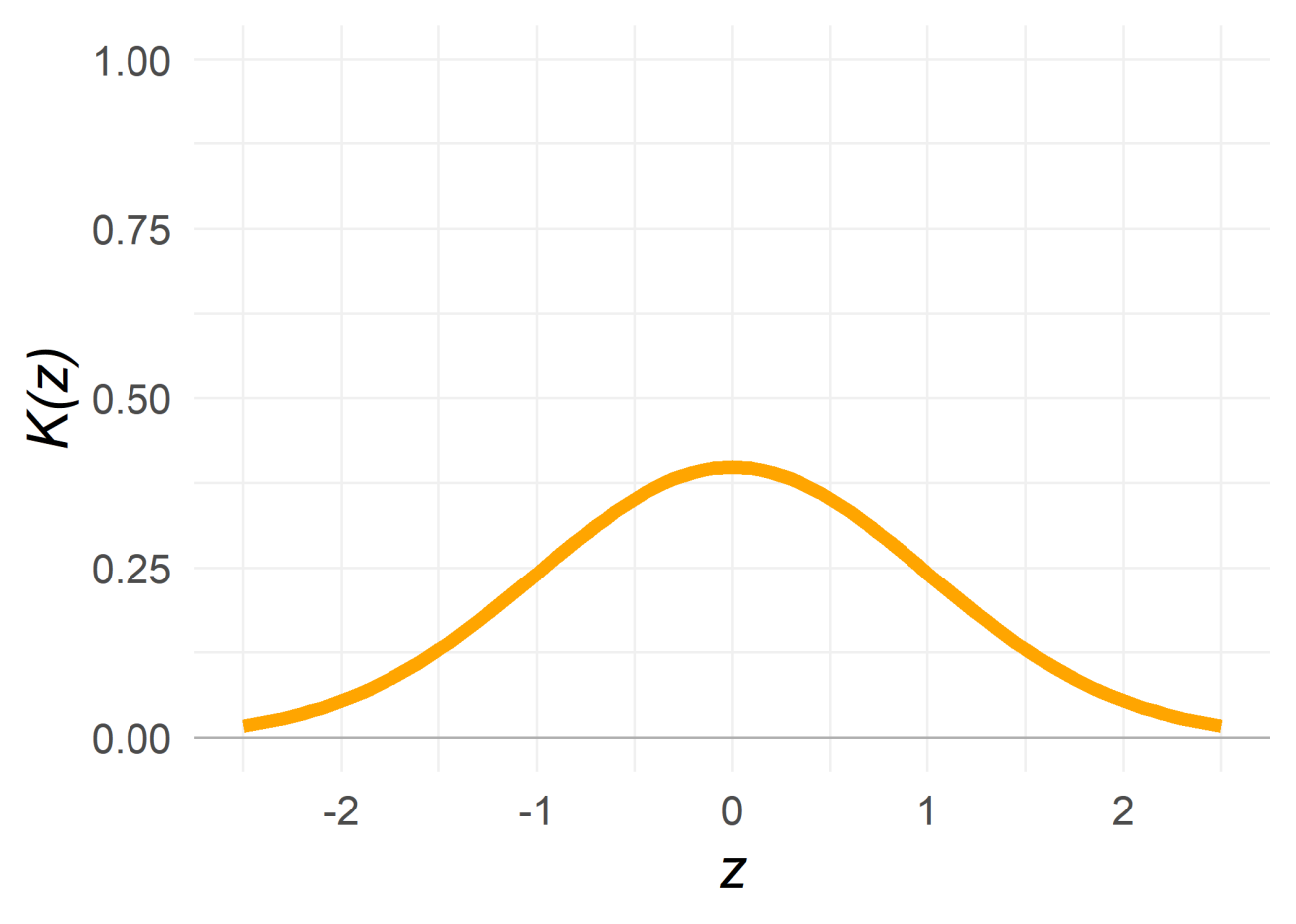
Kernels

And the **uniform kernel** with $K(z) = \frac{1}{2} \times \mathbb{I}(|z| < 1)$



Kernels

Or the **Gaussian kernel** $K(z) = (2\pi)^{-1/2} \exp\left(-z^2/2\right)$



Aside on Kernels

Kernel functions are good for more than just matching.

You will most commonly see/use them smoothing out densities—providing a smooth, moving-window average.

E.g. R's (`ggplot2`'s) smooth, density-plotting function `geom_density()`.

`geom_density()` defaults to `kernel = "gaussian"`, but you can specify many other kernel functions (including `"epanechnikov"`).

You can also change the `bandwidth` argument. The default is a bandwidth-choosing function called `bw.nrd0()`.

Matching

More recent techniques approach matching in one of two ways

1. As a **subset selection process**

- Use algorithms to produce (weighted) subsets where treatment is unassociated with (un)observed covariates
  - Treated units paired with control units; unpaired units are dropped (stratification)
- Reduce reliance on correct model specification in subsequent regressions (Ho et al. 2007)
- A non-parametric pre-processing approach

Matching

More recent techniques approach matching in one of two ways

2. As a **counterfactual imputation method**

- Pair units
- Impute the missing potential outcome for each unit using the observed outcomes of paired units
- Well-understood theoretical properties (Abadie and Imbens 2006, 2016)

Matching

If taking a **subset selection** approach, there are many options:

- **Nearest Neighbor (Greedy)**: pair to control unit(s) closest on a distance measure
- **Optimal Pair**: pair to control unit(s) by optimizing an overall criterion
- **Optimal Full**: Assign each treated + control units to a subclass
  - each subclass: 1 T and $\geq$1 C, or $\geq$1 T and 1 C
  - Minimizes the within-subclass distances
  - Weights chosen by subclass membership
  - **Generalized Full**: fast clustering algorithm to improve comp. time
- **Genetic**: find scaling factors for each covariate, then do NN

Matching

If taking a **subset selection** approach, there are many options:

- **Exact**: create subclasses using unique covariate values
  - Exactly balances covariate distributions
  - **Coarsened Exact**: exact but on binned covariate values
- **Cardinality/Profile**: "all or nothing" weighting method
  - Finds the largest sample that satisfies provided balance constraints
  - Solves a mixed integer program (**Zubizerreta, Paredes, and Rosenbaum 2014**)
  - Cardinality: balance on mean differences in matched sample, T + C samples same size
  - Profile: balance on mean difference between T and target distribution
  - Could then also create pairs to improve precision (**Zubizerreta, Paredes, and Rosenbaum 2014**)

Matching

Woah, that's a lot of options. How do we choose?

Ho et al. (2007) recommend the following:

- Try a matching method
  - For ATE: optimal, generalized full, subclassification, or profile
  - For ATT/ATC: any
- Assess balance
  - Distributional balance, not just mean balance
- If it yields poor balance or a too-low sample size, try another method
- Attempt to come as close as possible to perfect balance

Canonical Synthetic Control

Canonical Synthetic Control

The **canonical synthetic control method** feels a lot like if **event study** and a **matching estimator** got together and had a kid.

- Originated in **Abadie and Gardeazabal (2003) AER**, refined and extended in **Abadie, Diamond, and Hainmueller (2010) JASA**
- Developed for **comparative case studies:** one aggregate unit exposed to treatment/intervention

The gist: compare post-treatment outcome evolution in treated group to a **synthetic control** unit constructed to match on

- Pre-trends in the outcome variable $\mathbf{Y}_i$,
- and optionally: Covariates $\mathbf{X}_i$ or $\mathbf{X}_{it}$

Canonical Synthetic Control

Synthetic control overcomes one of the key issues of studying aggregate units: **few, poor-counterfactual controls**

- Policy interventions often happen at an aggregate level (i.e. state, country)
- Aggregate/macro data are often easy to obtain

However,

- Finding a valid counterfactual with coarse, aggregate units can be difficult
- Control group selection is ad hoc, leading to *researcher degrees of freedom*

Canonical Synthetic Control

Formally:

- Suppose you have data for $J + 1$ units
  - Treated unit: $j = 1$
  - "Donor Pool": all $j = 2, \ldots, J + 1$ units
- Data span T periods, with T_0 periods prior to treatment

For each unit, we observe

1. The outcome of interest Y_{jt}
2. A set of k predictors of the outcome, $X_{1j}, \ldots, X_{kj}$
   - May include pre-intervention values of Y_{jt}
   - Must be .hi-blue[unaffected by the intervention]

Canonical Synthetic Control

Formally:

For each unit j, let Y_{jt}^N be the potential response **without intervention**, with Y_{1t}^I the potential response **under intervention** for the treated unit

- For unit "one" with $t > T_0$, we have $Y_{1t} = Y_{1t}^I$

Under this setup, the effect of the policy in period t is given by

$$\tau_{1t} = Y_{1t}^I - Y_{1t}^N$$

Policy Evaluation Challenge: how to estimate Y_{1t}^N, the unobserved counterfactual?

Canonical Synthetic Control

A: Construct a **synthetic control** as a weighted average of units in the donor pool.

Let $W = (\omega_2, \ldots, \omega_{J+1})'$ be a $J \times 1$ vector of weights.

For a given W, the synthetic control counterfactual is

$$\hat{Y}_{1t}^{N} = \sum_{j=1}^{J+1} \omega_j Y_{jt}$$

and

$$\hat{\tau}_{1t} = Y_{1t}^{I} - \hat{Y}_{1t}^{N}$$

Choosing Weights

Weights are designed to **avoid extrapolation**

- $\omega_j \geq 0 \ \ \forall \ j$
- $\sum_{j=1}^{J+1} \omega_j = 1$
- Ensures synthetic control is located within the convex hull of donor units (based purely on observed data)

We will choose the ω_j so that the synthetic control **matches pre-intervention values for the treated unit of predictors for the outcome variable**.

Identification Under SCM

matches pre-intervention values for the treated unit of predictors for the outcome variable.

The whole idea behind SCM and its nice statistical properties relies on our ability to **approximate relevant unobservables**.

Arkhangelsky and Hirshberg (2023) formalize that identification of canonical SCM *and* the extensions we'll talk through next time holds under **two main conditions:**

- **Sequential Exogeneity:** conditional on observed pre-treatment information and permanent unobserved characteristics, treatment is independent of future counterfactual outcomes.
  - **Best case:** conditional mean is a linear function of past outcomes (e.g. random walk)

Identification Under SCM

matches pre-intervention values for the treated unit of predictors for the outcome variable.

The whole idea behind SCM and its nice statistical properties relies on our ability to **approximate relevant unobservables**.

Arkhangelsky and Hirshberg (2023) formalize that identification of canonical SCM *and* the extensions we'll talk through next time holds under **two main conditions:**

- **Retrievability:** when the number of pre-treatment periods is sufficiently large, relevant unobservables can be precisely recovered.

Identification Under SCM

If **Sequential Exogeneity** and **Retrievability** hold, then

- SCM estimators are consistent and asymptotically normal, even when treatment assignment is a deterministic function of the outcome (permanent and/or time-varying)
  - DiD: *not* consistent here

Identification Under SCM

Situations where **SCM Breaks Down**:

- Heterogeneity in the persistence of time-varying shocks
  - $\Rightarrow$ pre-treatment information insufficient to construct weights
  - $\Rightarrow$ potentially overcome by including unit-level measures of persistence
- Violation of strict exogeneity
  - $\Rightarrow$ adoption decisions are correlated with time-varying component of outcome (conditional on permanent unobserved heterogeneity)

Choosing Weights

Formally, choose weights W^* that minimize

$$||X_1 - X_0 W|| = (\sum_{h=1}^{k} \nu_h (X_{h1} - \omega_2 X_{h2} - \ldots - \omega_{J+1} X_{hJ+1})^2)^{1/2}$$

- Positive constants $\nu_1 \ldots \nu_k$ reflect the **relative importance** put on predictors $1, \ldots k$
  - **Abadie, Diamond, and Hainmueller (2010)**: select $\nu_1 \ldots \nu_k$ to minimize **mean square prediction error (MSPE)** for some set of pre-intervention periods

Choosing Weights

Formally, choose weights W^* that minimize

$$||X_1 - X_0 W|| = (\sum_{h=1}^{k} \nu_h (X_{h1} - \omega_2 X_{h2} - \ldots - \omega_{J+1} X_{hJ+1})^2)^{1/2}$$

- Positive constants $\nu_1 \ldots \nu_k$ reflect the **relative importance** put on predictors $1, \ldots k$
- **Abadie, Diamond, and Hainmueller (2015)**: select $\nu_1 \ldots \nu_k$ via out-of-sample validation
  1. Divide pre-intervention period into .hi-medgrn[training] and .hi-purple[validation] periods
  2. Select a value of $V^* = \nu_1^* \ldots \nu_k^*$ that yields a small MSPE in the validation period
  3. Use resulting V^* to calculate optimal weights in the validation period

German Reunification

To see how this process works, let's see an example: **Impact of 1990 German Reunification on GDP**.

Let's load in some state-by-year data on GDP and other economic conditions:

```
deu ← haven::read_dta("data/repgermany.dta") %>%
    mutate_at(vars(index, year, gdp, infrate, trade, schooling,
                   invest60, invest70, invest80,
                   industry),
              as.numeric) %>%
    mutate_at(vars(country), as.character)
```

German Reunification

Let's load in some state-by-year data on GDP and other economic conditions:

```
head(deu)
```

```
## # A tibble: 6 × 11
##   index country  year   gdp infrate trade schooling invest60 invest70 invest
##   <dbl> <chr>   <dbl> <dbl>   <dbl> <dbl>     <dbl>    <dbl>    <dbl>    <db
## 1     1 USA      1960  2879   NA     9.69      43.8       NA       NA
## 2     1 USA      1961  2929    1.08  9.44      NA         NA       NA
## 3     1 USA      1962  3103    1.12  9.43      NA         NA       NA
## 4     1 USA      1963  3227    1.21  9.47      NA         NA       NA
## 5     1 USA      1964  3420    1.31  9.73      NA         NA       NA
## 6     1 USA      1965  3667    1.67  9.73      43.8       NA       NA
## # ℹ 1 more variable: industry <dbl>
```

German Reunification

How did West Germany GDP compare to OECD countries **prior** to reunification in 1990?

- *Spoiler:* that gap looks to be **growing**

German Reunification

What if we construct a "synthetic" West Germany to match on pre-unification predictors of economic growth?

- Pre-treatment outcome
  - GDP (average for 1981-1990)

- Covariates
  - Trade openness: Exports + Imports as % of GDP (average for 1981-1990)
  - Inflation Rate (average for 1981-1990)
  - Industry share of value-added (average 1971-1980)
  - Schooling: % of secondary school attained in the age 25+ population (5-year averages, 1970 and 1975)
  - Investment rate: ratio of real domestic investment (private + public) to real GDP (5-year average 1970)

German Reunification

Let's use the **tidysynth** package to do this in a *tidy* workflow

one hiccup: tidysynth was recently **removed from CRAN**[1]

```
install.packages("tidysynth")
```

```
## Error in contrib.url(repos, "source"): trying to use CRAN without setting a
```

[1] CRAN has automatic checks in place that sometimes remove packages because of seemingly arbitrary reasons. **tidysynth** was removed because it failed some automated HTML manual issues, not because of any issue with the methods the package uses.

Installing from Github

We can get around this pretty easily by remembering the **remotes** package we saw in lecture 1

- Allows installation of packages directly from
  - GitHub, GitLab, BitBucket
  - Local files
  - Specific package versions on CRAN

```
remotes::install_github("edunford/tidysynth")
pacman::p_load(tidysynth)
```

German Reunification

Let's use the **tidysynth** package to do this in a *tidy* workflow

First, let's set up the synthetic control object with `synthetic_control()`

```
synth_wg ← deu %>%
    synthetic_control(
      outcome = gdp, # outcome
      unit = country, # unit identifier
      time = year, # time identifier
      i_unit = "West Germany", # treated unit
      i_time = 1990, # treatment year
      generate_placebos = T # whether to generate placebos for inference
    )
```

German Reunification

Next, add the predictors with `generate_predictor()`

- Choose a time period to use for matching (`time_window`)
  - Default: full pre-intervention period
- Choose the variables to use
- Choose the summary method (e.g. `sum()`, `mean()`, `min()`, `sum(is.na())`)
- Choose the variable names for the new predictors

German Reunification

Adding the predictors with `generate_predictor()`:

- First: 1981-1990 averages for GDP, Trade Openness, and Inflation Rates

```
synth_wg ← synth_wg %>%
  generate_predictor(time_window = 1981:1990,
                     gdp_81_90 = mean(gdp, na.rm = T),
                     trade81_90 = mean(trade, na.rm = T),
                     infrate81_90 = mean(infrate, na.rm = T)
  ) %>%
  generate_predictor(time_window = 1971:1980,
                     industry_71_80 = mean(industry, na.rm = T)) %>%
  generate_predictor(time_window =c(1970, 1975),
                     schooling_70_75 = mean(schooling, na.rm = T)) %>%
  generate_predictor(time_window = 1980,
                     invest_70 = invest70)
```

German Reunification

Adding the predictors with `generate_predictor()`:

- Second: 1971-1980 average for Industry shares

```
synth_wg ← synth_wg %>%
  generate_predictor(time_window = 1981:1990,
                     gdp_81_90 = mean(gdp, na.rm = T),
                     trade81_90 = mean(trade, na.rm = T),
                     infrate81_90 = mean(infrate, na.rm = T)
  ) %>%
  generate_predictor(time_window = 1971:1980,
                     industry_71_80 = mean(industry, na.rm = T)) %>%
  generate_predictor(time_window =c(1970, 1975),
                     schooling_70_75 = mean(schooling, na.rm = T)) %>%
  generate_predictor(time_window = 1980,
                     invest_80 = invest80)
```

German Reunification

Adding the predictors with `generate_predictor()`:

- Mean of 1970-1975 and 1975-1980 secondary school attainment

```
synth_wg ← synth_wg %>%
  generate_predictor(time_window = 1981:1990,
                     gdp_81_90 = mean(gdp, na.rm = T),
                     trade81_90 = mean(trade, na.rm = T),
                     infrate81_90 = mean(infrate, na.rm = T)
  ) %>%
  generate_predictor(time_window = 1971:1980,
                     industry_71_80 = mean(industry, na.rm = T)) %>%
  generate_predictor(time_window =c(1970, 1975),
                     schooling_70_75 = mean(schooling, na.rm = T)) %>%
  generate_predictor(time_window = 1980,
                     invest_80 = invest80)
```

German Reunification

Adding the predictors with `generate_predictor()`:

- 1980-1985 ratio of (real) domestic investment to GDP

```
synth_wg ← synth_wg %>%
  generate_predictor(time_window = 1981:1990,
                     gdp_81_90 = mean(gdp, na.rm = T),
                     trade81_90 = mean(trade, na.rm = T),
                     infrate81_90 = mean(infrate, na.rm = T)
  ) %>%
  generate_predictor(time_window = 1971:1980,
                     industry_71_80 = mean(industry, na.rm = T)) %>%
  generate_predictor(time_window =c(1970, 1975),
                     schooling_70_75 = mean(schooling, na.rm = T)) %>%
  generate_predictor(time_window = 1980,
                     invest_80 = invest80)
```

German Reunification

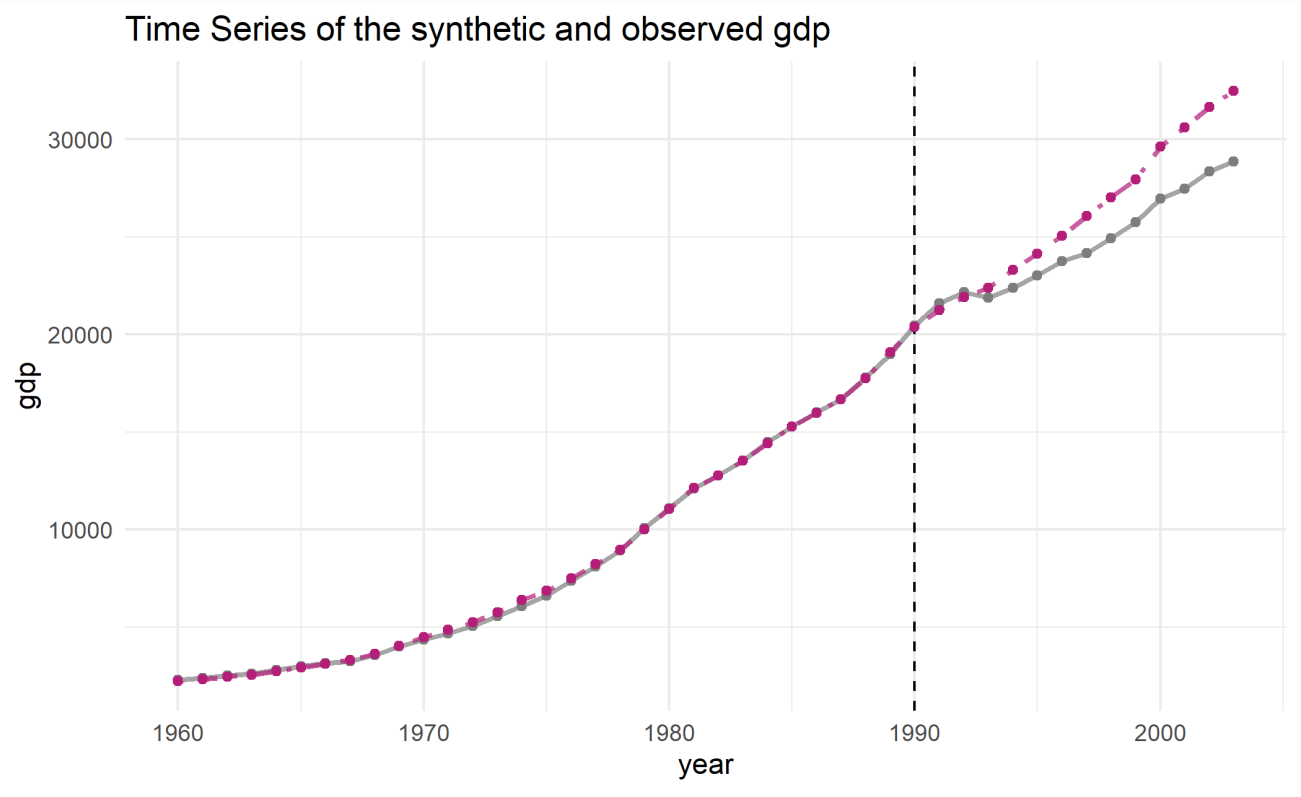
Next, generate weights with `generate_weights()`

```
wts ← synth_wg %>%
  generate_weights(optimization_window = 1981:1990)

# get variable weights
wt_vec ← wts[[7]][[1]] %>%
  select(weight) %>% as.vector() %>% unlist()
```

German Reunification

Finally, estimate the synthetic control and plot it

```
synth_control ← generate_control(wts)

plot_trends(synth_control)
```



German Reunification

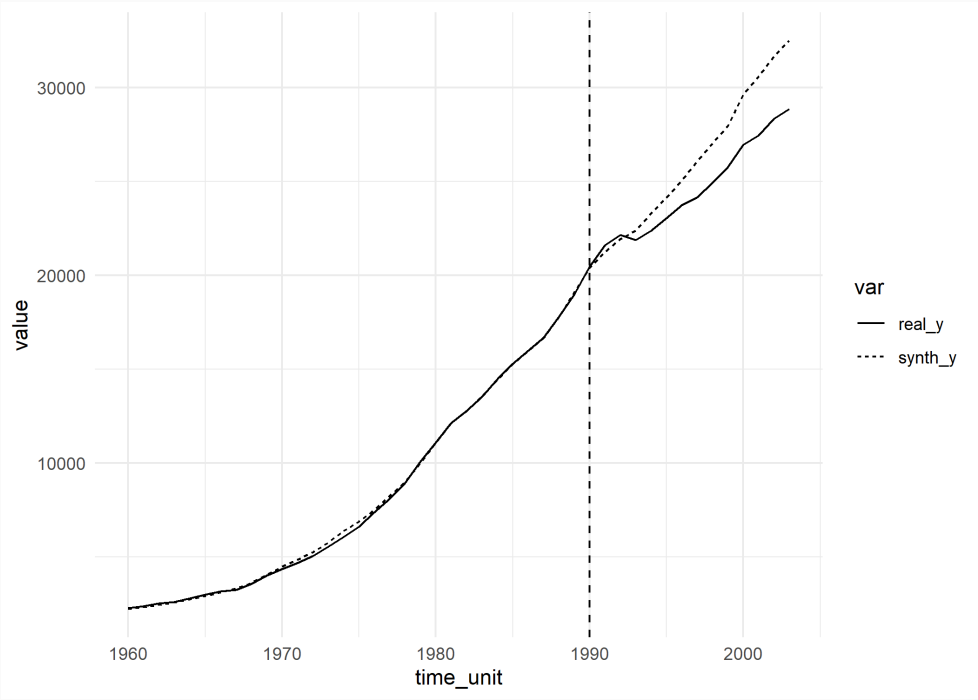
Alternatively, extract the synthetic control + treated unit values for manual plotting:

```
grab_synthetic_control(synth_control) %>% head()
```

```
## # A tibble: 6 × 3
##   time_unit real_y synth_y
##       <dbl>  <dbl>   <dbl>
## 1      1960   2284   2238.
## 2      1961   2388   2319.
## 3      1962   2527   2457.
## 4      1963   2610   2565.
## 5      1964   2806   2751.
## 6      1965   3005   2918.
```

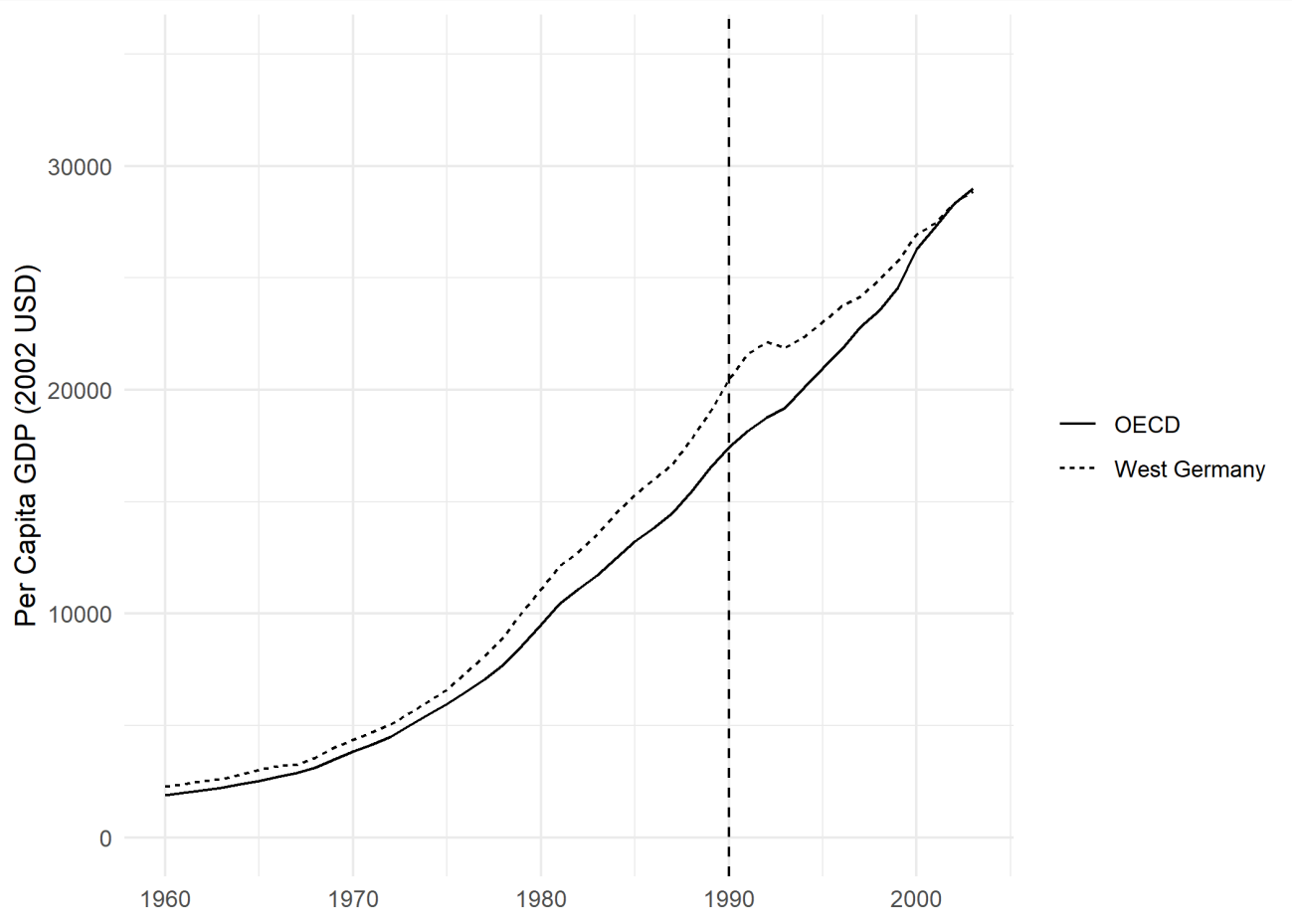
German Reunification

```
grab_synthetic_control(synth_control) %>%
  pivot_longer(cols = ends_with("y"), names_to = "var") %>%
  ggplot(aes(x = time_unit)) +
  geom_line(aes(y = value, linetype = var)) +
    geom_vline(aes(xintercept = 1990), linetype = "dashed") +
  theme_minimal()
```



German Reunification

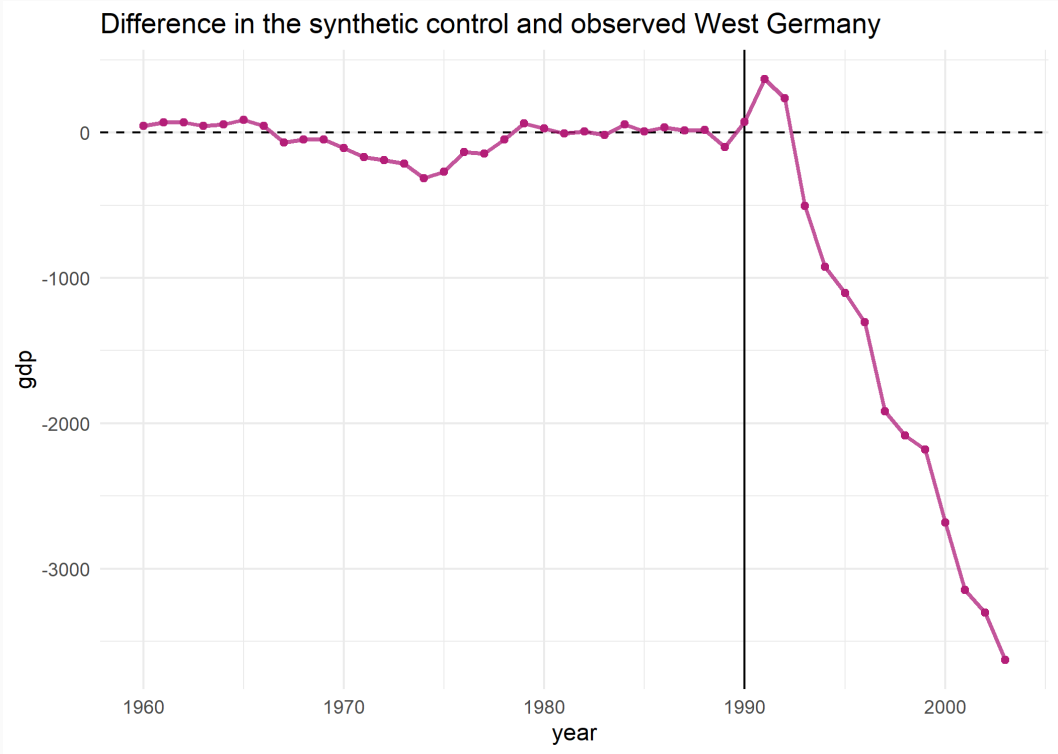
Comparing to the raw mean of OECD countries:



German Reunification

Alernatively we can plot the **difference** between West Germany and its synthetic control:

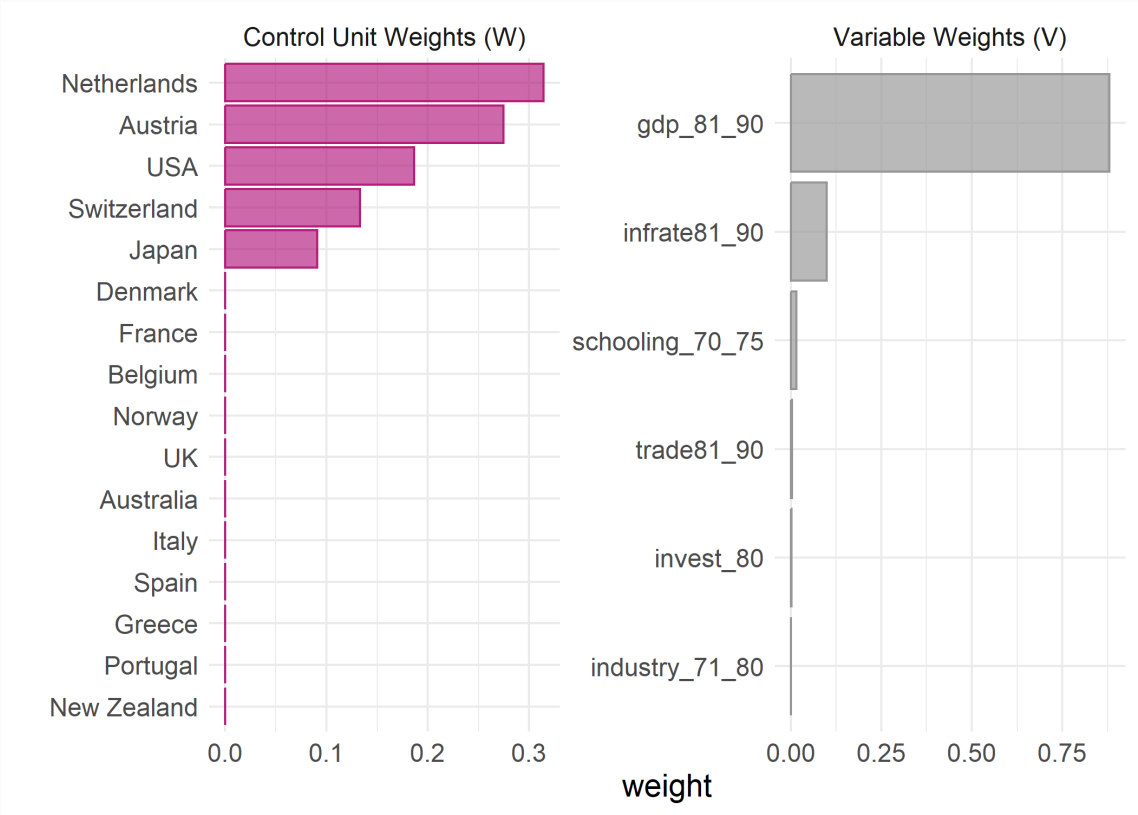
```
plot_differences(synth_control)
```



German Reunification

Looking at both the control unit and variable weights with `plot_weights()`:

```
synth_control %>%
  plot_weights()
```



German Reunification

Checking balance of real West Germany vs. Synthetic West Germany vs. Mean of OECD Countries with `grab_balance_table()`:

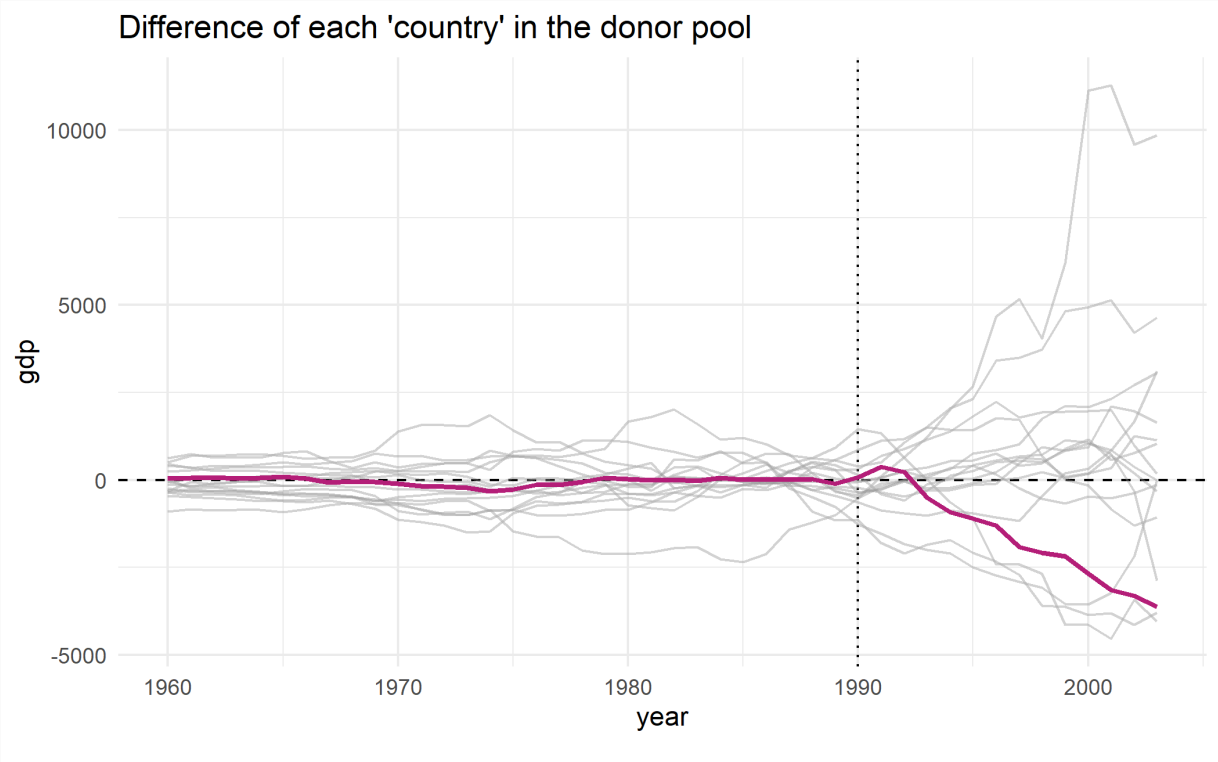
```
synth_control %>%
  grab_balance_table()
```

```
## # A tibble: 6 × 4
##   variable        West Germany synthetic_West Germany donor_sample
##   <chr>                  <dbl>                  <dbl>        <dbl>
## 1 gdp_81_90             15809.                 15800.       13669.
## 2 infrate81_90              2.59                   3.30         7.62
## 3 trade81_90               56.8                   69.1         59.8
## 4 industry_71_80           43.9                   37.1         36.9
## 5 schooling_70_75          51.9                   43.1         32.5
## 6 invest_80                27.0                   25.7         25.9
```

German Reunification

For inference, we repeat the same process as before with every unit in the donor pool.

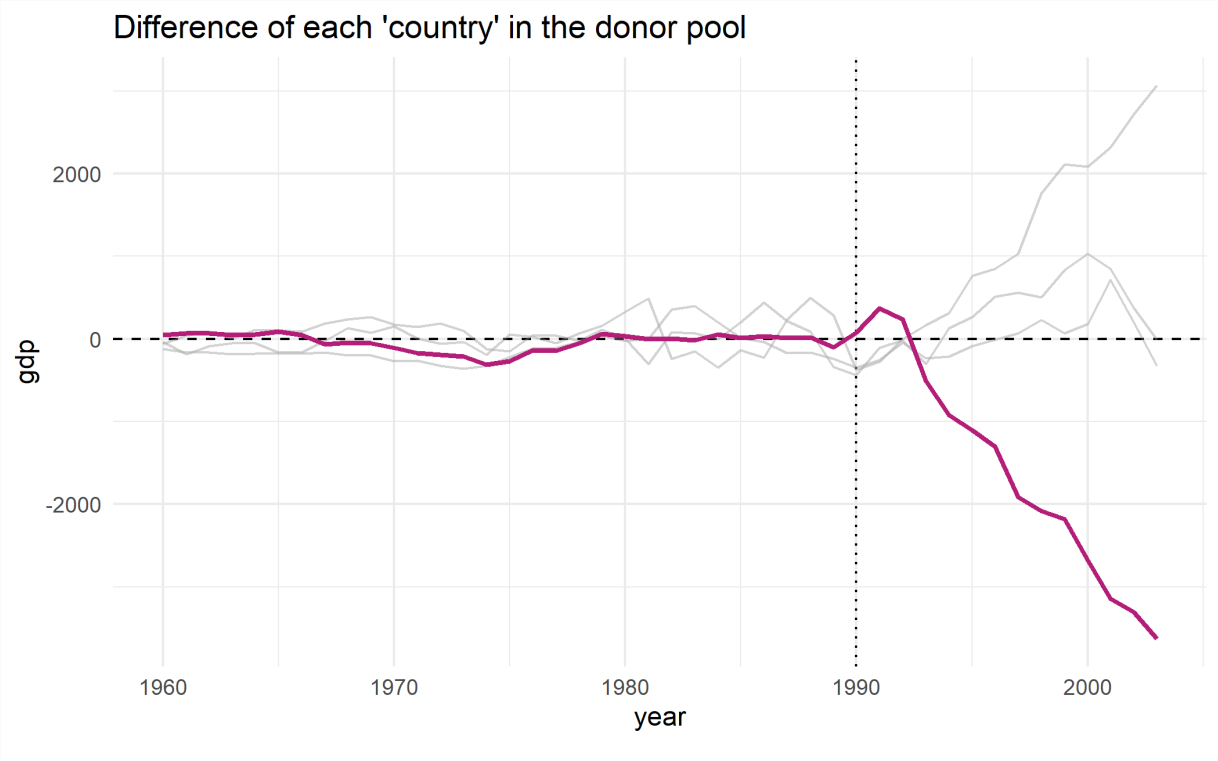
```
synth_control %>%
  plot_placebos(prune = FALSE)
```



German Reunification

By default, `plot_placebos()` hides the placebo controls with large MSPEs (here we only get 3)

```
synth_control %>%
  plot_placebos()
```



Inference

Finally, looking at inference:

```
wg_inf ← synth_control %>%
  grab_significance()
wg_inf
```

```
## # A tibble: 17 × 8
##    unit_name    type    pre_mspe post_mspe mspe_ratio  rank fishers_exact_pv
##    <chr>        <chr>      <dbl>     <dbl>      <dbl> <int>                <
##  1 West Germany Treated   12771.  4474135.    350.        1                0.
##  2 Norway       Donor    583094. 42826838.     73.4       2                0.
##  3 Australia    Donor     44373.  2816702.     63.5       3                0.
##  4 USA          Donor    331001. 12654685.     38.2       4                0.
##  5 New Zealand  Donor    388407.  9052988.     23.3       5                0.
##  6 Greece       Donor    410485.  6625701.     16.1       6                0.
##  7 Spain        Donor    122462.  1569470.     12.8       7                0.
##  8 Italy        Donor    297045.  2505509.      8.43      8                0.
##  9 Denmark      Donor     34078.   280217.      8.22      9                0.
## 10 Switzerland  Donor   1399771.  8117976.      5.80     10                0.
## 11 Netherlands  Donor    230567.  1094375.      4.75     11                0.
```

Inference

```
colnames(wg_inf)
```

```
## [1] "unit_name"           "type"                "pre_mspe"
## [4] "post_mspe"           "mspe_ratio"          "rank"
## [7] "fishers_exact_pvalue" "z_score"
```

Inference with synthetic controls is based on the **difference between pre and post-intervention MSPE** values.

Idea: if the synthetic control fits the observed data well (low pre-intervention MSPE), and diverges in the post-period (high post-period MSPE), then the intervention had a meaningful effect.

- If the intervention had *no* effect, the pre and post-period MSPE should be similar, with a ratio around 1
- If placebos fit the data as well as the treated unit, we can't reject the null of no treatment effect

Inference

Fisher's exact P-value is generated by first ranking ratios then dividing the rank of the case over the total

```
unique_countries ← unique(deu$country) %>% length()

# Fisher's P calculated as rank/total, so for West Germany (rank 1):
which(wg_inf$unit_name == "West Germany")/unique_countries
```

```
## [1] 0.05882353
```

Z-score is then the standardized RMSE ratios for all cases.

- Captures degree to which a particular case's RMSE ratio deviates from the **placebo distribution**

Choice of Predictors

One challenge remaining for the researcher is the **definition of predictors**

- Which predictors to use
- Which years to match on

  Ferman, Pinto, and Possebom (2020) go into great detail regarding how to properly select specifications of synthetic controls. Their punchline:
  - Models including more pre-treatment outcome lags as predictors are better at controlling for unobserved confounders
  - The possibilities for "specification searching" are higher with more pre-treatment periods used for matching
  - **Best:** present multiple results under common specifications
- If the result is robust to these different predictor choices, then the preferred specification isn't cherry-picked!

Table of Contents